0 0 4
server's answer: Upper Triangle Matrix

Exp 2:

server:

```
#include <stdio.h>
#include <string.h>
#include <stdlib.h>
#include <arpa/inet.h>
#include <unistd.h>

#define PORT 8080

void translate(char str[])
{
    struct
    {
        char shortf[20];
        char full[50];
    } dict[] =
    {
        {"tbh","to be honest"},
        {"sus","suspicious"},
        {"ig","I guess"},
        {"tbf","to be fair"},
        {"atm","at the moment"},
        {"irl","in real life"},
        {"lol","laughing out loud"},
        {"asap","as soon as possible"},
        {"omg","oh my God"},
        {"ttyl","talk to you later"},
        {"idk","I don't know"},
        {"nvm","never mind"},
        {"idc","I don't care"}
    };

    int size = sizeof(dict) / sizeof(dict[0]);

    char result[1024] = "";
    char temp[1024];
    strcpy(temp, str);
```

```
    char *token = strtok(temp, " ");
    while (token != NULL)
    {
        int found = 0;

        for (int i = 0; i < size; i++)
        {
            if (strcmp(token, dict[i].shortf) == 0)
            {
                strcat(result, dict[i].full);
                strcat(result, " ");
                found = 1;
                break;
            }
        }

        if (!found)
        {
            strcat(result, token);
            strcat(result, " ");
        }

        token = strtok(NULL, " ");
    }

    strcpy(str, result);
}

int main()
{
    int server_sock;
    char buffer[1024];

    struct sockaddr_in serverAddr, clientAddr;
    socklen_t len;

    server_sock = socket(AF_INET, SOCK_DGRAM, 0);

    serverAddr.sin_family = AF_INET;
    serverAddr.sin_port = htons(PORT);
    serverAddr.sin_addr.s_addr = INADDR_ANY;

    bind(server_sock, (struct sockaddr *)&serverAddr, sizeof(serverAddr));
```

```
    printf("UDP Server Running...\n");

    len = sizeof(clientAddr);

    recvfrom(server_sock, buffer, sizeof(buffer), 0,
            (struct sockaddr *)&clientAddr, &len);

    printf("Received: %s\n", buffer);

    translate(buffer);

    sendto(server_sock, buffer, strlen(buffer) + 1, 0,
           (struct sockaddr *)&clientAddr, len);

    close(server_sock);

    return 0;
}
```

client:

```
#include <stdio.h>
#include <string.h>
#include <arpa/inet.h>
#include <unistd.h>

#define PORT 8080

int main()
{
int sockfd;
char buffer[1024];

struct sockaddr_in serverAddr;

sockfd = socket(AF_INET, SOCK_DGRAM, 0);

serverAddr.sin_family = AF_INET;
serverAddr.sin_port = htons(PORT);
serverAddr.sin_addr.s_addr = inet_addr("127.0.0.1");

printf("Enter Sentence:\n");
fgets(buffer,sizeof(buffer),stdin);
```

```
sendto(sockfd,buffer,strlen(buffer)+1,0,
       (struct sockaddr*)&serverAddr,sizeof(serverAddr));

recvfrom(sockfd,buffer,sizeof(buffer),0,NULL,NULL);

printf("\nTranslated Sentence:\n%s\n",buffer);

close(sockfd);

return 0;
}
```

output:
//server
UDP Server Running...
Received: omg

//client
Enter Sentence:
omg
Translated Sentence:
oh my God

Exp 3:

server:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>
#include <pthread.h>
#define MAX_CLIENTS 10

int clients[MAX_CLIENTS];
int clientCount = 0;
pthread_mutex_t lock;

void broadcast(char message[], int sender)
{
   pthread_mutex_lock(&lock);
```